

CSE 219 / CSE 220 – Signals & Linear Systems

Copy-Paste Function & NumPy/Matplotlib Quick Reference

Part 1 — every function that could plausibly be asked for in the lab exam (time shift/scale/reverse, interpolation, even/odd decomposition, step/impulse synthesis, convolution, energy/power, system-property tests, and plotting helpers), organized by topic and ready to copy directly into your solution.

Part 2 — every NumPy and Matplotlib function that appears across your three reference documents, each with a 2-line usage note, organized by category for a fast lookup during the exam.

Built from: DSP Lab Exam Guide, Signals Coding Question Bank, and the Python Signals Lab Handnote.

Contents

PART 1 — Function Library

1. Time Axis & Basic Signal Builders
2. Interpolation (the single most important idea in the exam)
3. Continuous-Time Transforms (shift / scale / reverse / affine)
4. Even / Odd Decomposition
5. Discrete Sinusoid: Time Shift vs Phase Change
6. Step / Impulse Synthesis
7. Discrete-Time Index Operations (shift / reverse / resample)
8. Energy, Power, Convolution, Correlation
9. Testing System Properties (Linearity / Time-Invariance / Causality)
10. Plotting Helpers
11. General Python Patterns (occasionally tested)

PART 2 — NumPy / Matplotlib Quick Reference

- A. Array Creation
- B. Universal Functions (element-wise math)
- C. Aggregation / Reduction
- D. Boolean Masks & Comparisons
- E. Searching & Sorting
- F. Shape, Indexing & Structure
- G. Interpolation & Signal-Specific
- H. Misc / Constants
- I. Matplotlib — Plot Types
- J. Matplotlib — Layout & Decoration

PART 1 — Function Library

Every function below is copy-paste ready. Where the source documents disagreed on a convention (e.g. the direction of a discrete time shift), both variants are shown with a note on how to tell which one your paper wants.

1. Time Axis & Basic Signal Builders

make_axis_and_signal() — continuous grid + a starter signal

The two lines you type first in almost every solution.

```
def make_axis_and_signal():
    t = np.linspace(T_MIN, T_MAX, N)    # N points, endpoints included
    x = np.sin(2*np.pi*0.5*t) + 0.5*np.sin(2*np.pi*1.5*t)
    return t, x
```

make_index() — discrete integer axis

arange's stop is EXCLUSIVE, so pass 21 to include 20.

```
def make_index():
    n = np.arange(-20, 21)             # -20, -19, ..., 20
    return n
```

x_of_t_triangle(t) — triangular pulse, half-width 1 at t=0

Classic base signal for shift / scale demos.

```
def x_of_t_triangle(t):
    x = np.zeros_like(t)
    m = np.abs(t) <= 1.0
    x[m] = 1.0 - np.abs(t[m])
    return x
```

x_of_t_gaussian_cosine(t) — smooth bump, good for time-scaling

Asymmetric-looking so compression/stretch is visually obvious.

```
def x_of_t_gaussian_cosine(t):
    return np.exp(-t**2) * np.cos(2*np.pi*0.75*t)
```

x_of_t_piecewise_ramp(t) — step then down-ramp

Used in the worked $x(-3t+2)$ affine-transform example.

```
def x_of_t_piecewise_ramp(t):
    y = np.zeros_like(t)
    y[(t >= 0) & (t < 1)] = 1.0
    m = (t >= 1) & (t <= 2)
    y[m] = 2.0 - t[m]
    return y
```

x_of_t_mixed_even_odd(t) — deliberately neither even nor odd

Use this for even/odd decomposition demos so both parts are non-trivial.

```
def x_of_t_mixed_even_odd(t):
    tri = np.zeros_like(t); m = np.abs(t) <= 1.0
    tri[m] = 1.0 - np.abs(t[m])
    ramp = np.zeros_like(t); ramp[m] = t[m]
    return tri + 0.6 * ramp
```

2. Interpolation (the single most important idea in the exam)

interpolate_signal(t, x, q) — hand-rolled, midpoint-average form

Matches the rule $y(1)=0.5*(x(0)+x(1))$; identical to `np.interp` to $\sim 1e-16$.

```
def interpolate_signal(t, x, q):
    idx_r = np.clip(np.searchsorted(t, q, side="left"), 1, len(t)-1)
    idx_l = idx_r - 1
    tL, tR = t[idx_l], t[idx_r]
    xL, xR = x[idx_l], x[idx_r]
    w = np.where(tR != tL, (q - tL) / (tR - tL), 0.0)
    return xL + w * (xR - xL)      # == 0.5*(xL+xR) at a midpoint
```

interpolate_signal(t_org, x_org, t_query) — exact-hit + NaN-outside version

Keeps exact grid hits exact; returns NaN outside range (`plot()` skips NaNs).

```
def interpolate_signal(t_org, x_org, t_query):
    order = np.argsort(t_org)
    ts, xs = t_org[order], x_org[order]
    n = len(ts)
    idx = np.searchsorted(ts, t_query, side="left")
    idc = np.clip(idx, 0, n - 1)
    exact = (idx < n) & np.isclose(ts[idc], t_query)
    left = xs[np.clip(idx - 1, 0, n - 1)]
    right = xs[np.clip(idx, 0, n - 1)]
    res = np.where(exact, xs[idc], 0.5 * (left + right))
    outside = (t_query < ts[0]) | (t_query > ts[-1])
    return np.where(outside, np.nan, res)
```

True linear interpolation (weighted, not just the midpoint average)

Use if a question explicitly wants distance-weighted interpolation.

```
# i0 = idx - 1 (left neighbour), i1 = idx (right neighbour)
w = (t_query - ts[i0]) / (ts[i1] - ts[i0])  # 0 at left, 1 at right
val = (1 - w) * xs[i0] + w * xs[i1]
# built-in equivalent on a sorted grid:
val = np.interp(t_query, ts, xs)
```

np.interp one-liner — use this unless a question forbids it

`left=0.0, right=0.0` zero-fills outside the recorded window (default clamps instead).

```
y = np.interp(query_times, t, x, left=0.0, right=0.0)
```

Fixing the classic searchsorted IndexError

`searchsorted` can return `len(ts)` -- always clip before indexing.

```
idx = np.searchsorted(ts, t_query)
n = len(ts)
right = xs[np.clip(idx, 0, n - 1)]
left = xs[np.clip(idx - 1, 0, n - 1)]
```

3. Continuous-Time Transforms (shift / scale / reverse / affine)

time_shift(t, x, t0) — $y(t) = x(t - t_0)$

Positive t_0 delays (slides right); negative advances (slides left).

```
def time_shift(t, x, t0):
    q = t - t0
    return np.interp(q, t, x, left=0.0, right=0.0)
```

time_scale(t, x, a) — general $y(t) = x(a t)$

$a > 1$ compresses (narrower); $0 < a < 1$ stretches (wider).

```
def time_scale(t, x, a):
    q = a * t
    y = np.zeros_like(t)
    inr = (q >= t[0]) & (q <= t[-1])
    y[inr] = np.interp(q[inr], t, x) # or interpolate_signal(t,x,q[inr])
    return y
```

time_scale(t, x, k) — sub-scaling $y(t) = x(t / k)$

The exact A1/A2 task: query the SAMPLES, never re-call the analytic x_of_t .

```
def time_scale(t, x, k):
    return interpolate_signal(t, x, t / k)
```

time_reverse(x) — $y(t) = x(-t)$ on a symmetric grid

Only valid if t is symmetric about 0 (e.g. `linspace(-4,4,4001)`, odd N).

```
def time_reverse(x):
    return x[::-1].copy() # .copy() -> standalone, not an aliased view
```

time_reverse on an ASYMMETRIC grid

The array-flip shortcut only works on a symmetric grid; otherwise remap directly.

```
xr = np.interp(-t, t, x, left=0.0, right=0.0)
```

affine_transform(t, x, a, b) — general $y(t) = x(a t + b)$

Folds shift+reverse+scale into ONE remap/interpolation -- no error compounding.

```
def affine_transform(t, x, a, b):
    q = a * t + b
    y = np.zeros_like(t)
    inr = (q >= t[0]) & (q <= t[-1])
    y[inr] = np.interp(q[inr], t, x)
    return y
```

Hand-recipe for $x(at+b)$ (for the drawing / explanation part of the answer)

Order matters: doing shift-then-scale $\neq$ scale-then-shift.

```
# 1. shift first:      g1(t) = x(t + b)
# 2. reverse if a<0:   g2(t) = g1(-t)
# 3. scale by |a|:     g3(t) = g2(|a| t) = x(a t + b)
```

amp_combine(x, z, A, B) — $y(t) = A x(t) + B z(t)$

Amplitude only -- no interpolation needed, but x and z MUST share the same t .

```
def amp_combine(x, z, A, B):
    return A * x + B * z
```

4. Even / Odd Decomposition

even_odd_decompose(x)

Must be built from `time_reverse` per the spec; two element-wise lines.

```
def even_odd_decompose(x):
    xr = time_reverse(x)      # x(-t)
    xe = 0.5 * (x + xr)      # even part
    xo = 0.5 * (x - xr)      # odd part
    return xe, xo
```

Numeric proof checks (print these -- all should be ~0)

Reconstruction, evenness, oddness -- the three identities graders ask for.

```
print(np.max(np.abs((xe + xo) - x)))      # reconstruction
print(np.max(np.abs(time_reverse(xe) - xe))) # xe even
print(np.max(np.abs(time_reverse(xo) + xo))) # xo odd
```

5. Discrete Sinusoid: Time Shift vs Phase Change

sinusoid(n, A, Omega0, phi) — $x[n] = A \cos(\text{Omega0 } n + \text{phi})$

The base building block for all of Section 5.

```
def sinusoid(n, A, Omega0, phi):
    return A * np.cos(Omega0 * n + phi)
```

time_shift_sinusoid — CHECK WHICH CONVENTION YOUR PAPER USES

Convention 1 = a delay $x[n-n_0]$ (DSP Guide). Convention 2 = $x[n+n_0]$ (Question Bank/Handnote).

```
# Convention 1: DELAY, x[n - n0]
def time_shift_sinusoid(n, A, Omega0, phi, n0):
    return A * np.cos(Omega0 * (n - n0) + phi)

# Convention 2: x[n + n0]
def time_shift_sinusoid(n, A, Omega0, phi, n0):
    return sinusoid(n + n0, A, Omega0, phi)
```

phase_change_sinusoid(n, A, Omega0, phi, phi0)

Adds phi0 to the existing phase argument.

```
def phase_change_sinusoid(n, A, Omega0, phi, phi0):
    return sinusoid(n, A, Omega0, phi + phi0)
```

phi0_equiv — the EXACT phase that reproduces a given time shift

Sign must match whichever `time_shift_sinusoid` convention you used above.

```
# if using x[n - n0]: phi0_equiv = -Omega0 * n0
# if using x[n + n0]: phi0_equiv = +Omega0 * n0
phi0_equiv = Omega0 * n0 # <-- flip sign to match your convention
```

mse(a, b) — mean squared error helper

Never compare float signals with `==`; use this instead.

```
def mse(a, b):
    return float(np.mean((a - b) ** 2))
```

Best-integer-shift search (Part B pattern)

Finds the integer k whose time-shift best matches an arbitrary phase change.

```
best_k, best_err = None, None
for k in range(-12, 13):
    e = mse(time_shift_sinusoid(n, A, Omega0, phi, k), x_phase)
    if best_err is None or e < best_err:
        best_err, best_k = e, k
```

discrete_period(Omega0, max_N=1000) — smallest integer period, or None

Periodic iff $\Omega_0 N$ is a multiple of 2π for some integer N .

```
def discrete_period(Omega0, max_N=1000):
    for N in range(1, max_N + 1):
        ratio = Omega0 * N / (2 * np.pi)
        if np.isclose(ratio, round(ratio)):
            return N
    return None
```

6. Step / Impulse Synthesis

unit_step(n) — $u[n]$

1 for $n \geq 0$, else 0, as an integer array.

```
def unit_step(n):
    return (n >= 0).astype(int)
```

unit_impulse(n) — $\delta[n]$

1 only at $n == 0$.

```
def unit_impulse(n):
    return (n == 0).astype(int)
```

shifted_step(n, n0) — $u[n - n_0]$

Positive n_0 delays the step (moves right); negative advances it (moves left).

```
def shifted_step(n, n0):
    return (n - n0 >= 0).astype(int)
```

gated_exponential(n, a) — $a^n * u[n]$

Multiplying by the boolean mask zeroes out the $n < 0$ part.

```
def gated_exponential(n, a):
    return (a ** n) * (n >= 0)
```

rect_pulse(n, a, b) — 1 on $[a, b]$ inclusive, else 0

Combine two conditions with $\&$ (parenthesised), never Python's and .

```
def rect_pulse(n, a, b):
    return ((n >= a) & (n <= b)).astype(int)
```

Weighted sum of shifted impulses — build any $x[n] = \sum c_k \delta[n-nk]$

Example: $x[n] = 2\delta[n-1] - 3\delta[n-2] + 2\delta[n-4]$.

```
x = (2*unit_impulse(n-1)
     - 3*unit_impulse(n-2)
     + 2*unit_impulse(n-4)).astype(float)
```

Impulse from step (first difference)

$\delta[n] = u[n] - u[n-1]$; prepend keeps the same length.

```
delta = np.diff(u, prepend=u[0])
```

Step from impulse (running sum)

`cumsum` accumulates left to right, turning a single spike into a step.

```
u = np.cumsum(delta)
```

Step built from a sum of K delayed impulses

Demonstrates $u[n] = \sum_{k=0}^K \delta[n-k]$ for n up to K .

```
K = 30
acc = np.zeros_like(n)
for k in range(K + 1):
    acc = acc + unit_impulse(n - k)
# vectorized alternative:
acc = ((n >= 0) & (n <= K)).astype(int)
```

7. Discrete-Time Index Operations (shift / reverse / resample)

shift(x, n0) — $y[n] = x[n - n0]$, zero-padded, same length

`np.roll` wraps around instead -- NOT the same as a finite-signal time shift.

```
def shift(x, n0):
    y = np.zeros_like(x)
    if n0 >= 0:
        y[n0:] = x[:len(x)-n0] if n0 < len(x) else y[n0:]
    else:
        y[:n0] = x[-n0:]
    return y
```

reverse_seq(x) — $x[-n]$ for an arbitrary stored index

Same one-liner as `time_reverse`; named separately for non-symmetric indices.

```
def reverse_seq(x):
    return x[::-1].copy()
```

Combined reversal + shift, e.g. $y[n] = x[-n + 2]$

Rewrite as $x[-(n-2)]$: shift the argument by 2, THEN reverse.

```
# want y[n] = x[-n + 2] = x[-(n - 2)]
xr = reverse_seq(x)    # x[-n]
y = shift(xr, 2)       # x[-(n-2)] = x[-n+2]
```


downsample(x, k) — keep every k-th sample

$x[0], x[k], x[2k], \dots$

```
def downsample(x, k):  
    return x[::k]
```

upsample(x, k) — insert k-1 zeros between samples

Output length is $k * \text{len}(x)$.

```
def upsample(x, k):  
    y = np.zeros(len(x) * k, dtype=x.dtype)  
    y[::k] = x  
    return y
```

8. Energy, Power, Convolution, Correlation

continuous_energy_power(t, x)

Riemann-sum energy (version-proof: no `np.trapz/np.trapezoid` needed).

```
def continuous_energy_power(t, x):  
    dt = t[1] - t[0]  
    E = np.sum(np.abs(x)**2) * dt  
    P = E / (t[-1] - t[0])  
    return E, P
```

discrete_energy_power(x)

P divides by the SAMPLE COUNT ($x.size$), matching $n2-n1+1$ for an inclusive range.

```
def discrete_energy_power(x):  
    E = np.sum(np.abs(x)**2)  
    P = E / x.size  
    return E, P
```

energy(x) / average_power(x) — simple standalone versions

Plain sum-of-squares / mean-of-squares, returned as float.

```
def energy(x):  
    return float(np.sum(x ** 2))  
  
def average_power(x):  
    return float(np.mean(x ** 2))
```

classify(x) — heuristic energy-signal vs power-signal

On a finite window this is only illustrative (the true test is a limit as $N \rightarrow \infty$).

```
def classify(x):  
    E = energy(x); P = average_power(x)  
    return "energy" if P < 1e-6 and E < np.inf else "power"
```

convolve(x, h) — full linear convolution

Output length is $\text{len}(x) + \text{len}(h) - 1$. `mode='same'/'valid'` for other lengths.

```
def convolve(x, h):  
    return np.convolve(x, h, mode="full")
```

Convolution with a unit impulse leaves a signal unchanged

The sifting property, verified numerically.

```
x = np.array([2.0, -1.0, 3.0, 0.5])
d = np.array([1.0])
y = np.convolve(x, d, mode="full")
print(np.allclose(y, x)) # True
```

moving_average(x, L) — boxcar smoothing filter

Convolving with a length-L box of height 1/L, same-length output.

```
def moving_average(x, L):
    h = np.ones(L) / L
    return np.convolve(x, h, mode="same")
```

cross_corr(x, y) — full cross-correlation

Auto-correlation is cross_corr(x, x); its peak sits at zero lag.

```
def cross_corr(x, y):
    return np.correlate(x, y, mode="full")
```

9. Testing System Properties (Linearity / Time-Invariance / Causality)

is_linear(S, x1, x2, a, b)

Checks $S(ax_1 + bx_2) \approx aS(x_1) + bS(x_2)$. Passing is strong evidence, not proof.

```
def is_linear(S, x1, x2, a, b):
    lhs = S(a * x1 + b * x2)
    rhs = a * S(x1) + b * S(x2)
    return bool(np.allclose(lhs, rhs))
```

is_time_invariant(S, x, n0)

Shifting the input by n_0 should shift the output by n_0 ; ignore zero-fill edge samples.

```
def is_time_invariant(S, x, n0):
    lhs = S(shift(x, n0))
    rhs = shift(S(x), n0)
    return bool(np.allclose(lhs[n0:], rhs[n0:]))
```

depends_on_future(S, x, i) — causality probe

Perturbs a strictly-future sample; True means the system is non-causal at i .

```
def depends_on_future(S, x, i):
    x2 = x.copy()
    if i + 1 < len(x):
        x2[i + 1] += 1.0
    return not np.isclose(S(x)[i], S(x2)[i])
```

10. Plotting Helpers

plot_pair(t, x, y, title) — continuous x(t) vs y(t)

Standard decorated two-curve plot with zero-axes drawn in.

```
def plot_pair(t, x, y, title):
    plt.figure(figsize=(9, 4))
    plt.plot(t, x, label="x(t)")
    plt.plot(t, y, label="y(t)", linestyle="--")
    plt.title(title); plt.xlabel("t"); plt.ylabel("Amplitude")
    plt.axhline(0, color="k", lw=0.8); plt.axvline(0, color="k", lw=0.8)
    plt.grid(True); plt.legend()
```

plot_three(t, x, xe, xo) — original + even + odd

The standard C1/C2-style even/odd figure.

```
def plot_three(t, x, xe, xo):
    plt.figure()
    plt.plot(t, x, label="x(t)")
    plt.plot(t, xe, label="xe(t)")
    plt.plot(t, xo, label="xo(t)")
    plt.title("Even-Odd Decomposition")
    plt.xlabel("t"); plt.ylabel("Amplitude")
    plt.grid(True); plt.legend()
```

stem_plot(ax, n, x, label) — single discrete series

stem returns 3 artists; hide the baseline for a cleaner look.

```
def stem_plot(ax, n, x, label):
    ml, sl, bl = ax.stem(n, x, label=label)
    bl.set_visible(False)
    ax.grid(True, alpha=0.3); ax.set_xlabel("n"); ax.set_ylabel("Amplitude")
```

stem_pair(n, x1, x2, l1, l2) — two discrete series overlaid

Used to visually confirm a time-shift equals a phase change.

```
def stem_pair(n, x1, x2, l1, l2):
    fig, ax = plt.subplots(figsize=(9, 4))
    for x, lab in [(x1, l1), (x2, l2)]:
        _, _, base = ax.stem(n, x, label=lab)
        base.set_visible(False)
    ax.set_xlabel("n"); ax.set_ylabel("Amplitude")
    ax.legend(); ax.grid(True, alpha=0.3)
```

2x2 dashboard skeleton (line / bar / scatter / stacked-bar)

Practice-Problem-4 style; fixes the 3 classic bugs (width vs color, size vs s, list + vs array +).

```

fig, axs = plt.subplots(2, 2, figsize=(14, 10))
axs[0,0].plot(months, north, linestyle="-", color="blue", label="North")
axs[0,0].plot(months, south, linestyle="--", color="green", label="South")
axs[0,0].plot(months, central, linestyle=":", color="red", label="Central")
axs[0,0].legend(); axs[0,0].grid(True)

axs[0,1].bar(branches, values, color=["blue","green","red"]) # color is KEYWORD
axs[0,1].grid(True, axis="y")

axs[1,0].scatter(months, north, color="blue", s=60) # s = size, not "size"

north_q = north.reshape(4,3).sum(axis=1) # NumPy arrays add element-wise
south_q = south.reshape(4,3).sum(axis=1)
axs[1,1].bar(q, north_q, color="blue", label="North")
axs[1,1].bar(q, south_q, bottom=north_q, color="green", label="South")

fig.suptitle("Overall Title", fontsize=16)
plt.tight_layout(rect=[0,0,1,0.96])
plt.show()

```

11. General Python Patterns (occasionally tested)

Employee-style class — class attr vs instance attr, `__str__`

self.name etc. are per-object; company_name is shared (like a Java static field).

```

class Employee:
    company_name = "ACME" # class attribute (shared)
    def __init__(self, name, emp_id):
        self.name = name # instance attributes (per object)
        self.employee_id = emp_id
        self.salaries = []
    def add_salary(self, s): self.salaries.append(s)
    def average_salary(self): return sum(self.salaries)/len(self.salaries)
    def highest_salary(self): return max(self.salaries)
    def annual_income(self): return sum(self.salaries)
    def __str__(self):
        return f"Employee Name : {self.name}"

```

dict accumulator pattern (Java HashMap-style)

The standard group-and-total-by-key idiom.

```

summary = {}
for name, sev, tt in visits:
    if name not in summary:
        summary[name] = {"time": 0, "sev": 0}
    summary[name]["time"] += tt
    summary[name]["sev"] += sev

```

Sort / pick best with a tie-break key

Negate for descending; add the name as a tie-breaker.

```

top = min(summary, key=lambda k: (-summary[k]["sev"], k))

```

PART 2 — NumPy / Matplotlib Quick Reference

Every built-in function used anywhere in your three source documents, grouped by category. Each entry is a signature plus a 2-line usage note — scan for the function name, read the note, move on.

A. Array Creation

<code>np.array(list)</code>	Builds an ndarray from a Python list/nested list. dtype is inferred unless given explicitly.
<code>np.zeros(N) / np.zeros((r,c))</code>	All-zero array of the given shape, float64 by default (like calloc).
<code>np.zeros_like(a)</code>	Zeros with the SAME shape and dtype as a. The standard 'blank canvas' for building signals.
<code>np.ones(shape)</code>	All-ones array of the given shape.
<code>np.full(N, val)</code>	Array of the given shape filled entirely with val.
<code>np.arange(start, stop, step)</code>	Integers from start up to but NOT including stop. Never use a float step here.
<code>np.linspace(start, stop, num)</code>	num evenly spaced points INCLUDING both endpoints. Use for continuous time axes.

B. Universal Functions (element-wise math)

<code>np.sin(a), np.cos(a), np.tan(a)</code>	Element-wise trig on a whole array. Never use math.sin on an array (it only accepts scalars).
<code>np.exp(a), np.sqrt(a), np.log(a)</code>	Element-wise exponential, square root, and natural log.
<code>np.abs(a)</code>	Element-wise absolute value. Use <code>np.abs(x)**2</code> (not <code>x**2</code>) for energy so complex signals work too.
<code>np.clip(a, lo, hi)</code>	Clamps every element into [lo, hi], element-wise. Like <code>std::clamp</code> applied to a whole array.
<code>np.where(cond, A, B)</code>	Vectorized ternary: picks from A where cond is True, else from B, element-wise.
<code>np.round(a, k)</code>	Rounds every element to k decimal places.

C. Aggregation / Reduction

<code>a.sum() / np.sum(a)</code>	Total of all elements; pass axis=0 (columns) or axis=1 (rows) to reduce along one direction only.
<code>a.mean() / np.mean(a)</code>	Average of all elements, or per-axis with axis=...
<code>a.max() / a.min()</code>	Largest / smallest element, overall or per axis.
<code>np.argmax(a) / np.argmin(a)</code>	Index of the largest / smallest element (per axis if axis= given).
<code>a.std()</code>	Standard deviation of the elements.

`np.argmax(np.abs(t - v))`

Idiom: index of the sample whose time/value is closest to v. Handy for spot-checks.

D. Boolean Masks & Comparisons

`a > v, a == v, a <= v ...`

Element-wise comparison; returns a boolean array (a 'mask'), not a single True/False.

`&, |, ~`

AND / OR / NOT for NumPy boolean arrays. ALWAYS parenthesise; never use Python's and/or/not on arrays.

`np.isin(a, b)`

Element-wise test of whether each element of a appears anywhere in b.

`np.isclose(a, b) /
np.allclose(a, b)`

Float-safe (near-)equality test. Never compare floating-point arrays with ==.

E. Searching & Sorting

`np.searchsorted(sorted_a,
q, side=)`

Insertion index that keeps sorted_a sorted; the core primitive behind every interpolation function.

`np.argsort(a)`

Indices that would sort a in ascending order (does not sort a itself).

`np.sort(a)`

Returns a new sorted copy of a; the original is untouched.

F. Shape, Indexing & Structure

`a.shape / a.size / a.dtype`

Metadata: dimensions tuple, total element count, and element type.

`a.reshape(r, c)`

A view of a with a new shape (same underlying data, same total element count).

`a[::-1]`

A reversed VIEW of a (aliases the same memory). Call .copy() if you need an independent array.

`a.copy()`

An independent deep copy of a; safe to mutate without affecting the original.

`np.vstack([a, b, ...])`

Stacks 1-D arrays as rows of a new 2-D array.

`a.astype(int) /
a.astype(float)`

Casts to a different dtype, e.g. turning a boolean mask into 0/1 integers.

G. Interpolation & Signal-Specific

`np.interp(q, xp, fp, left=,
right=)`

Linear interpolation on a sorted grid xp/fp, evaluated at query points q. THE key exam function.

`np.convolve(x, h, mode=)`

Discrete convolution; mode='full' (default length), 'same', or 'valid'.

`np.correlate(x, y, mode=)`

Cross-correlation of x and y; auto-correlation is np.correlate(x, x, ...).

`np.diff(a, prepend=)`

First difference between consecutive elements; prepend keeps the output the same length.

`np.cumsum(a)`

Running/cumulative sum along the array; turns an impulse into a step.

`np.roll(a, k)`

Circularly shifts elements by k, WRAPPING around -- not the same as a zero-filled time shift.

H. Misc / Constants

<code>np.pi</code>	The constant pi = 3.14159...; used everywhere in sinusoid/frequency formulas.
<code>np.random.*</code>	Random number generation; rarely required in this course but occasionally used for test signals.

I. Matplotlib — Plot Types

<code>plt.figure(figsize=(w,h))</code>	Starts a new figure of the given size in inches. Call once per new plot window.
<code>plt.plot(x, y, label=, linestyle=, color=)</code>	Line plot connecting samples; use for continuous-time $x(t)$, never for discrete $x[n]$.
<code>ax.stem(n, x, label=)</code>	Lollipop plot for discrete $x[n]$; returns (markerline, stemlines, baseline) -- hide the baseline for a clean look.
<code>plt.scatter(x, y, s=, c=)</code>	Scatter plot of discrete points; s sets marker SIZE (not 'size').
<code>plt.bar(names, vals, color=, bottom=)</code>	Bar chart; the 3rd POSITIONAL argument is width, so always pass color= by keyword.
<code>plt.hist(data, bins=)</code>	Histogram of one or more data series.

J. Matplotlib — Layout & Decoration

<code>plt.subplots(rows, cols, figsize=)</code>	Creates a figure plus a grid of Axes, indexed like a matrix: <code>axs[row, col]</code> .
<code>plt.title / xlabel / ylabel(text)</code>	Labels the current axes: chart title, x-axis label, y-axis label.
<code>plt.legend()</code>	Displays a legend built from each plotted series' label= argument.
<code>plt.grid(True, axis=)</code>	Toggles gridlines; axis='y' or axis='x' restricts gridlines to one direction only.
<code>plt.axhline(0) / plt.axvline(0)</code>	Draws a horizontal/vertical reference line, e.g. the $y=0$ or $t=0$ axis, to show symmetry.
<code>plt.xlim(a,b) / plt.ylim(a,b)</code>	Clips the visible axis range to [a, b].
<code>fig.suptitle(text, fontsize=)</code>	One overall title spanning an entire multi-panel figure.
<code>plt.tight_layout(rect=)</code>	Auto-spaces subplots so titles/labels don't overlap; use <code>rect=[0,0,1,0.96]</code> to leave room for suptitle.
<code>plt.show()</code>	Renders the figure on screen. REQUIRED at the end -- nothing appears without it.
<code>plt.savefig(path, dpi=)</code>	Writes the current figure to an image file instead of (or as well as) displaying it.

Exam-Day Quick Checklist

Boilerplate to type first (from memory)

```
import numpy as np
import matplotlib.pyplot as plt
T_MIN, T_MAX, N = -4.0, 4.0, 4001
t = np.linspace(T_MIN, T_MAX, N)    # continuous grid
# n = np.arange(-20, 21)           # discrete grid
```

The transform decision tree

1. Touches TIME ($x(t-t_0)$, $x(at)$, $x(-t)$, $x(at+b)$)? Build q = remap, then $\text{np.interp}(q, t, x, \text{left}=0, \text{right}=0)$, zero-fill out-of-range. Never shift raw indices.
2. Touches AMPLITUDE only ($Ax+Bz$)? Plain element-wise $A*x+B*z$, no interpolation, same grid required.
3. Reversal on a symmetric grid: $x[::-1].\text{copy}()$. Otherwise: $\text{np.interp}(-t, t, x, \text{left}=0, \text{right}=0)$.
4. Discrete signal: stem plot, integer arange, no interpolation for integer shifts.

Ten traps, condensed

- 1 $x(t - t_0)$ shifts RIGHT (delay), not left.
- 2 Time scaling acts on time, not amplitude; $a > 1$ compresses.
- 3 Fractional source index $\Rightarrow$ must interpolate; there is no $x[0.5]$.
- 4 Use $\&$ / $|$ / $\sim$ with parentheses, never and/or on arrays.
- 5 Slices are views (aliased); use $\text{.copy}()$ when in doubt.
- 6 Compare floats with np.allclose / MSE, never $==$.
- 7 Discrete power denominator is $n_2 - n_1 + 1$ ($x.\text{size}$) -- watch the fencepost.
- 8 np.arange 's stop is exclusive; np.linspace 's stop is inclusive.
- 9 axs from $\text{subplots}(2,2)$ is a 2-D array -- index $\text{axs}[r, c]$.
- 10 Energy uses $\text{np.abs}(x)**2$ (complex-safe); $\text{stem}()$ returns 3 artists -- hide the baseline.

Sanity print before you submit

```
i0 = np.argmin(np.abs(t))          # index nearest t = 0
print("y(0) =", y[i0], " x(0) =", x[i0])
print("reconstruction:", np.max(np.abs((xe + xo) - x)))    # ~1e-16 means correct
```